Projet BMC

Point de départ : un serveur clé-valeur minimal en C

Ce document présente le serveur clé-valeur fourni comme base du projet. L'objectif du projet est de reproduire l'approche de l'article **BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing** (Ghigoff et al., NSDI 2021) : placer devant un serveur clé-valeur un cache écrit en eBPF, qui répond aux requêtes directement depuis le pilote réseau.

Le document décrit successivement l'objectif du projet, l'utilisation du serveur fourni, son architecture interne, puis ses limites volontaires et la progression attendue.

1. Contexte et objectif du projet

Memcached est un cache clé-valeur en mémoire très utilisé. L'article BMC montre que, sous une charge de petites requêtes GET en UDP, l'essentiel du temps CPU n'est pas passé dans Memcached lui-même mais dans la pile réseau de Linux et dans les appels système. Une partie importante de ce coût vient de la contention sur le verrou de la file de réception de l'unique socket UDP.

BMC répond à ce problème en deux temps :

- **MemcachedSR** : une variante de Memcached où chaque thread possède sa propre socket UDP grâce à l'option `SO_REUSEPORT`, ce qui supprime la contention sur la file unique ;
- **BMC** : un cache placé dans le noyau, écrit en eBPF et attaché au hook **XDP** du pilote réseau. Il répond aux GET qui touchent le cache avant même que le paquet n'atteigne la pile réseau. Les SET (en TCP) invalident les entrées du cache, et les réponses sortantes de Memcached, interceptées au hook **TC egress**, le remplissent.

Plutôt que de partir de Memcached, dont le code est volumineux, vous partez d'un serveur volontairement simple, `kvstore`, qui parle le même protocole. Les programmes eBPF qu'ils écriront fonctionneront donc aussi bien devant `kvstore` que devant un vrai Memcached.

2. Le serveur kvstore

2.1 Fichiers fournis

Fichier	Rôle
`kvstore.c`	le serveur (environ 570 lignes de C, sans dépendance externe)
`Makefile`	compilation avec `gcc -O2 -Wall -Wextra` et `-pthread`
`test_kvstore.py`	10 tests fonctionnels (SET et DELETE en TCP, GET en TCP et en UDP)

2.2 Protocole pris en charge

Le serveur implémente un sous-ensemble du protocole texte de Memcached, en TCP et en UDP sur le même port (11211 par défaut) :

Commande	Réponse
`get <clé> [<clé> ...]`	`VALUE <clé> <flags> <taille>` puis la donnée, pour chaque clé trouvée, puis `END`
`set <clé> <flags> <exptime> <taille> [noreply]` suivi de la donnée	`STORED`
`delete <clé> [noreply]`	`DELETED` ou `NOT_FOUND`
`quit`	fermeture de la connexion
toute autre commande	`ERROR`

Les clés sont limitées à 250 octets (limite du protocole Memcached) et les valeurs à 1 Mo. Le champ `exptime` est accepté mais ignoré. En UDP, chaque datagramme commence par l'en-tête Memcached de 8 octets, et la requête comme la réponse doivent tenir dans un seul datagramme (au plus 1400 octets pour la réponse).

2.3 Compilation et utilisation

```
make
./kvstore [-p port] [-t nb_threads_udp] [-v]

# exemple : 4 threads UDP, port par défaut
./kvstore -t 4 &
python3 test_kvstore.py 127.0.0.1 11211
```

Pour tester à la main avec `nc`, il faut ajouter l'en-tête de 8 octets aux requêtes UDP :

```
printf 'set foo 0 0 3\r\nbar\r\n' | nc -q1 localhost 11211
printf '\x00\x01\x00\x00\x00\x01\x00\x00get foo\r\n' | nc -u -w1 localhost 11211
```

3. Architecture du code

3.1 Vue d'ensemble

Le programme lance deux serveurs sur le même port. Le thread principal accepte les connexions TCP et crée un thread par connexion (`tcp_conn_thread`). En parallèle, N threads (option `-t`, 4 par défaut) lisent tous la même socket UDP (`udp_thread`). Les deux chemins aboutissent à la même fonction de traitement du protocole, `process_request`, qui accède à la table de hachage.

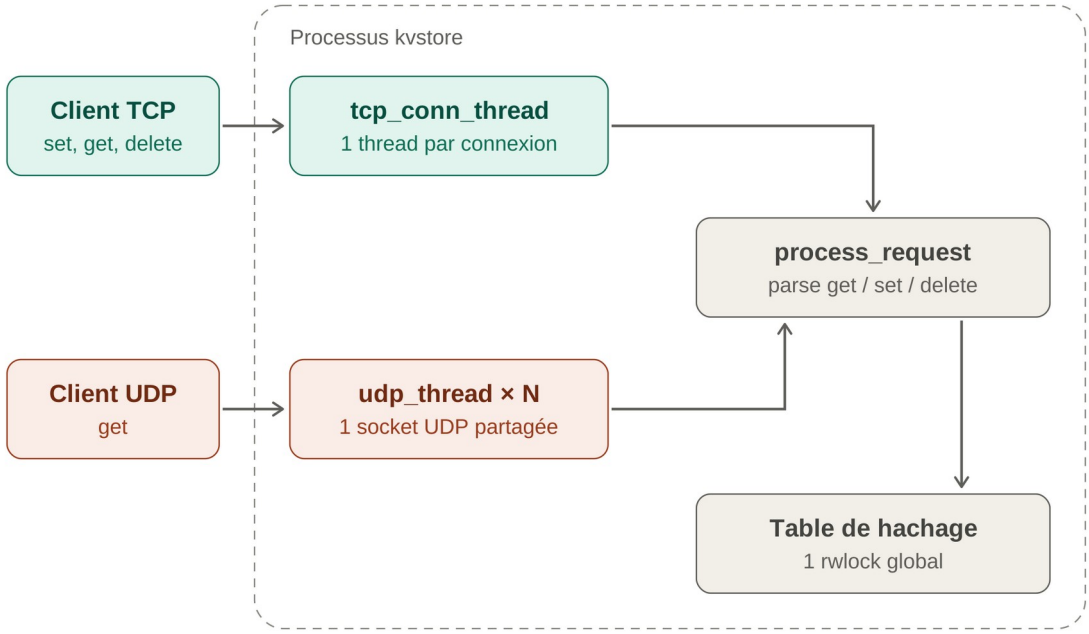


Figure 1 — *architecture générale de kvstore*

3.2 La table de hachage

Chaque paire clé-valeur est stockée dans une `struct item`. Le tableau flexible `data[]` placé en fin de structure permet d'allouer l'en-tête, la clé et la valeur en un seul appel à `malloc`.

```
struct item {
    struct item *next;   // élément suivant dans la même case
    uint32_t hash, flags, klen, vlen;
    char data[];         // la clé, puis la valeur
};
```

La table contient 2^20 cases (environ un million). La clé est hachée avec **FNV-1a 32 bits**, la même fonction que BMC : un programme eBPF qui calcule le même hachage tombe sur la même case, ce qui facilite le débogage. Les collisions sont résolues par **chaînage**.

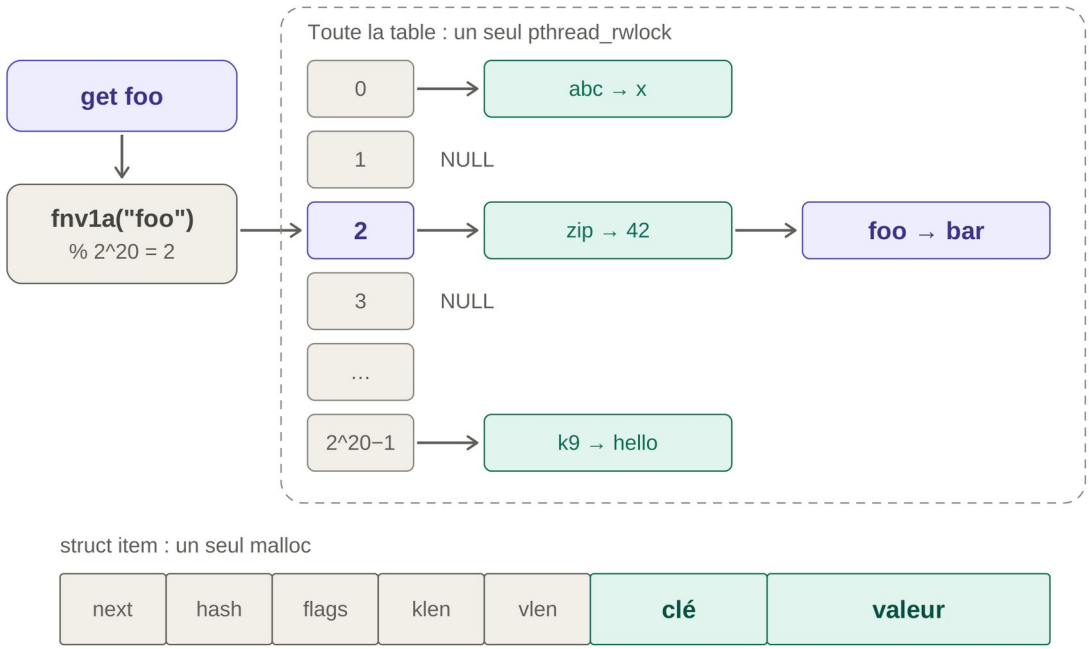


Figure 2 — *la table de hachage et la disposition mémoire d'un élément*

La fonction `lookup_slot` renvoie l'adresse du pointeur qui désigne l'élément, plutôt que l'élément lui-même. Ce double pointeur permet d'insérer, de remplacer ou de supprimer sans traiter à part la tête de liste.

- `kv_get` prend le verrou en lecture et **copie** la valeur dans un tampon. Renvoyer un pointeur serait incorrect : un `set` concurrent pourrait libérer l'élément juste après.
- `kv_set` prépare le nouvel élément hors du verrou, prend le verrou en écriture, échange les pointeurs, relâche le verrou, puis libère l'ancien élément. La section critique reste très courte.
- `kv_delete` décroche l'élément de sa chaîne de la même façon.

Toute la table est protégée par **un seul** `pthread_rwlock` : plusieurs lecteurs peuvent travailler en parallèle, mais une écriture bloque tout le monde.

3.3 Le tampon de sortie

`struct buf` est un tableau d'octets dont la capacité double lorsqu'il est plein. Toutes les réponses y sont accumulées puis envoyées en une seule fois : une requête TCP contenant plusieurs commandes (pipelining) ne provoque qu'un seul `send`.

3.4 Le parseur : process_request

La fonction reçoit des octets bruts et traite **une seule commande** au début du tampon. Elle renvoie l'une de trois valeurs, ce qui la rend indépendante du transport :

Valeur de retour	Signification	Action de l'appelant
`DONE`	commande traitée, `*consumed` octets lus	avance et traite la suivante
`NEED_MORE`	commande incomplète	attend d'autres octets (TCP)

Valeur de retour	Signification	Action de l'appelant
`CLOSE`	erreur grave ou `quit`	ferme la connexion

En TCP, une commande peut arriver en plusieurs morceaux, et plusieurs commandes peuvent arriver d'un coup. La fonction cherche d'abord la fin de ligne `\r\n`, copie la ligne, puis la découpe en mots avec `strtok_r` (la version réentrante, indispensable avec plusieurs threads). Pour un `set`, elle vérifie en plus que la donnée et son `\r\n` final sont arrivés ; si la taille annoncée est invalide, elle ferme la connexion, car il serait impossible de savoir combien d'octets ignorer pour se resynchroniser.

Le tampon `valbuf`, qui reçoit la copie des valeurs, est déclaré `__thread` : chaque thread possède le sien, sans partage ni verrou.

3.5 Le chemin TCP

Chaque connexion dispose d'un tampon d'entrée assez grand pour une ligne et la plus grande valeur possible. La boucle de `tcp_conn_thread` :

1. lit ce qui arrive avec `recv` et l'ajoute au tampon ;
2. appelle `process_request` tant qu'elle renvoie `DONE` ;
3. envoie en une fois toutes les réponses accumulées ;
4. décale vers le début du tampon ce qui n'a pas encore été consommé, avec `memmove`.

3.6 Le chemin UDP

C'est la partie que BMC court-circuite. Chaque datagramme commence par l'en-tête Memcached de 8 octets, en ordre réseau : identifiant de requête, numéro de séquence, nombre total de datagrammes et un champ réservé. Le serveur ignore les requêtes en plusieurs datagrammes. Il construit la réponse avec le même identifiant, la séquence 0 et un seul datagramme, puis y ajoute le résultat de `process_request`. Si la réponse dépasse 1400 octets, elle est remplacée par un message d'erreur.

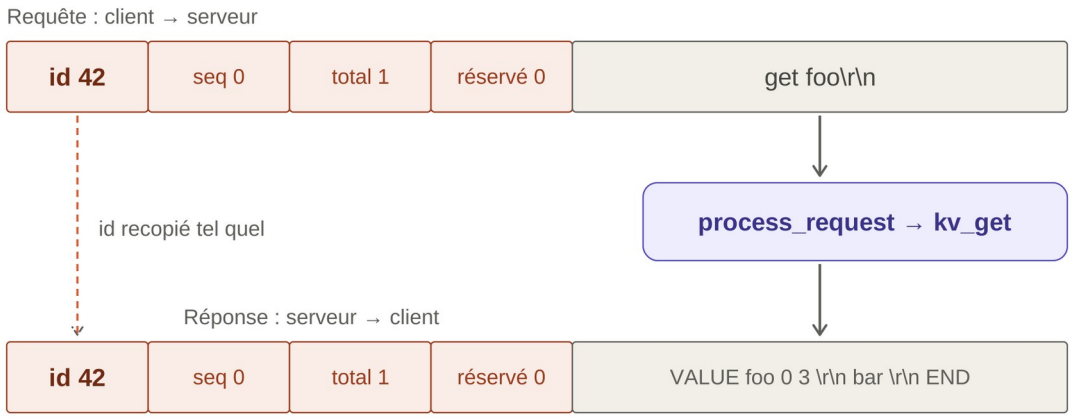


Figure 3 — *format d'une requête GET en UDP et de sa réponse*

C'est exactement ce que les programmes `prepare_packet` et `write_reply` de BMC devront reproduire dans le noyau : ce code sert de référence de format.

3.7 La fonction main

`main` lit les options, ignore `SIGPIPE` (sans quoi écrire vers un client TCP déconnecté tuerait le processus), alloue la table, crée les deux sockets, lance les threads UDP, puis boucle sur `accept()`.

4. Limites volontaires et lien avec l'article

Le serveur est volontairement naïf. Ses faiblesses correspondent aux étapes du projet. La plus importante apparaît en suivant un GET UDP de la carte réseau jusqu'au thread qui le traite : tous les paquets traversent la pile réseau, puis convergent vers une seule file de socket partagée par tous les threads.

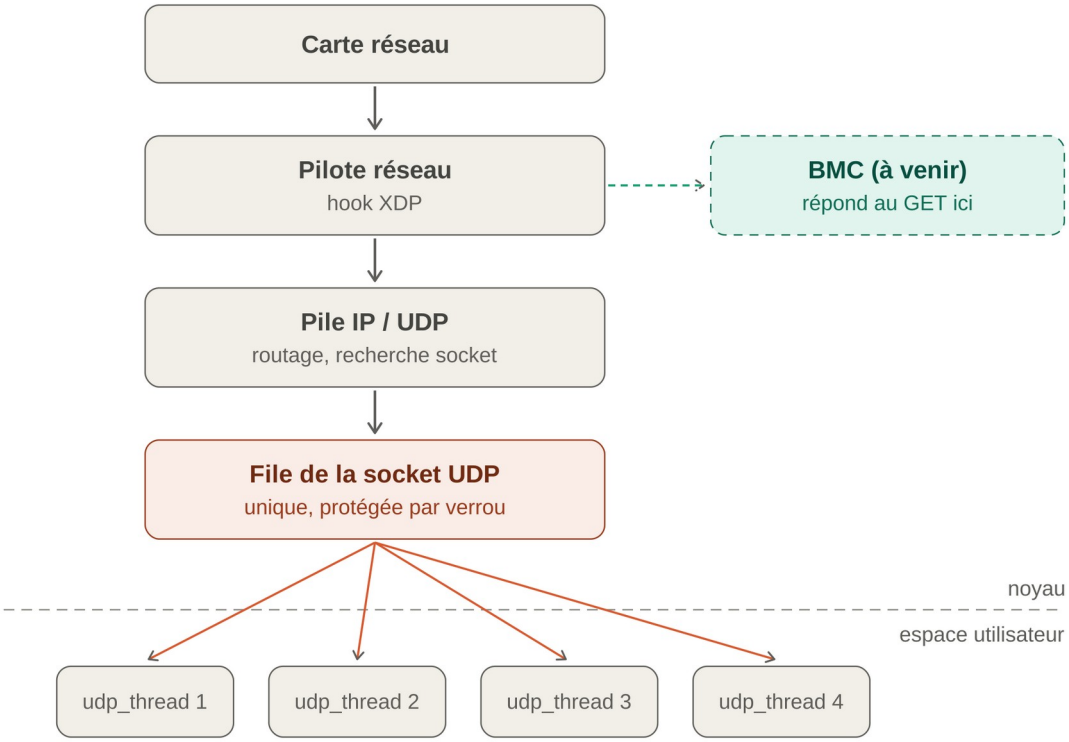


Figure 4 — *trajet d'un GET UDP dans le noyau et point d'insertion de BMC*

Choix naïf	Effet	Évolution attendue
Une seule socket UDP pour tous les threads	contention sur le verrou de la file de la socket (Tableau 1 de l'article)	`SO_REUSEPORT` : une socket par thread (équivalent de MemcachedSR)
Un verrou global sur la table	chaque `set` bloque tous les `get`	verrous par case ou par segment
GET UDP traité en espace utilisateur	traversée complète de la pile réseau et deux appels système par requête	cache XDP en eBPF (BMC)
SET reçus en TCP	aucun impact ici	point d'invalidation du cache BMC
Pas d'expiration ni d'éviction	la mémoire croît sans limite	hors du périmètre du projet

5. Progression suggérée

1. **Mesurer la référence** : écrire ou utiliser un générateur de charge UDP (popularité des clés selon une loi de Zipf, paramètre 0,99) et mesurer le débit de `kvstore` en faisant varier le nombre de threads. Profiler avec `perf` pour retrouver la contention sur la socket.

2. **Ajouter SO_REUSEPORT** : une socket UDP par thread. Mesurer à nouveau et comparer avec la Figure 3 de l'article.
3. **Filtre XDP** : un premier programme eBPF qui reconnaît les GET UDP destinés au port du serveur et les laisse passer (`XDP_PASS`). Vérifier le filtrage avec des compteurs dans une map.
4. **Cache et réponse** : table eBPF à correspondance directe, hachage FNV-1a, réécriture du paquet en réponse (en-têtes Ethernet, IP, UDP et en-tête Memcached) et renvoi avec `XDP_TX`.
5. **Mise à jour** : programme TC egress qui intercepte les réponses du serveur pour remplir le cache.
6. **Invalidation** : traitement des SET TCP dans le programme XDP pour invalider l'entrée correspondante.
7. **Évaluation** : débit, latence, taux de succès du cache, comparaison avec et sans BMC, et charge ne visant pas le cache (grandes valeurs).